

GSFC UNIVERSITY
FACULTY OF TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

A MAJOR PROJECT REPORT ON
DATABASE ENGINE IMPLEMENTATION AND RELATIONAL ALGEBRA COMPILER

Submitted by:
DIAGNOSTIC STUDENT
Enrollment No: CSE2026-004

Under the Guidance of:
Prof. Jane Guidance

August 2026

CHAPTER 1: INTRODUCTION AND DATABASE ARCHITECTURE

1.1 Introduction

The design and implementation of relational databases forms the core of modern software architecture.

Our database system utilizes a structured relational database architecture where data is organized into tables or relations.

1.2 System Architecture

The system consists of a query parser, relational algebra optimizer, storage engine, and a concurrency controller.

This architecture guarantees that the database engine can manage tables, indexes, and tuples efficiently.

We represent schema definitions and database architecture elements cleanly to provide transactional integrity.

CHAPTER 2: COMPILING RELATIONAL ALGEBRA OPERATORS

2.1 Formal Definition of Operators

Relational Algebra Selection and Projection Operators are the principal operators utilized to prune and retrieve data.

The selection operator filters rows based on a given predicate, whereas the projection operator filters columns.

By executing projection, we construct a new relation containing only the requested attributes.

By combining selection and projection operators, queries can be optimized before execution.

2.2 Algebraic Optimization

Query optimization involves rewriting relational algebra expressions into equivalent and more efficient forms.

This compiler translates SQL Select Queries into relational algebra expression trees for direct optimization.

CHAPTER 3: SQL SELECT QUERIES AND SCHEMA DESIGN

3.1 SQL Query Parsing

Modern database systems process SQL Select Queries and Joins by parsing text strings into abstract syntax trees.

We have implemented support for inner joins, left joins, and cross products within our relational compiler.

Each join is mapped into an equivalent cartesian product followed by a selection operator.

3.2 Schema Design and Normalization

To prevent anomalies, tables are normalized using Functional Dependencies.

We analyze functional dependencies to ensure the schema is in Third Normal Form (3NF) or Boyce-Codd Normal Form (BCNF).

Designing relational schemas using functional dependencies guarantees schema consistency and prunes redundancy.